Environment-1 (CoAP + HTTP)

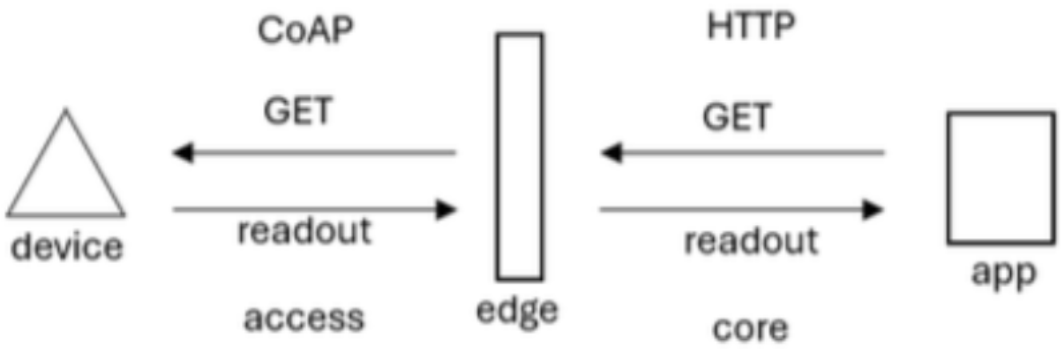
Setup Overview

For this environment, I established a network environment to analyze the communication between an embedded device and an application using CoAP and HTTP protocols. The setup consisted of three machines:

1. **Device Machine**: Implemented a CoAP server using libcoap that provides time data
2. **Edge Machine**: Created a bridge between CoAP and HTTP using a Node.js proxy
3. **Application Machine**: Used HTTP clients to request data from the Edge Machine

The communication flow followed this pattern:

- Application → Edge (HTTP GET request)
- Edge → Device (CoAP GET request)
- Device → Edge (CoAP response)
- Edge → Application (HTTP response)



I(a). Capture Wireshark Traces

```
pruthvi@pruthvi:~$ curl http://172.16.249.138:8080/time
Apr 24 16:50:24
pruthvi@pruthvi:~$ curl http://172.16.249.138:8080/time
Apr 24 17:07:23
```

Fig: HTTP requests from Application Machine to Edge Machine (172.16.249.138:8080/time), showing successful retrieval of time data. Two responses ("Apr 24 16:50:24" and "Apr 24 17:07:23") demonstrate the complete communication flow where Edge Machine processes the requests, retrieves data from Device Machine via CoAP, and returns it via HTTP.

Device Machine (IP: 172.16.249.140)

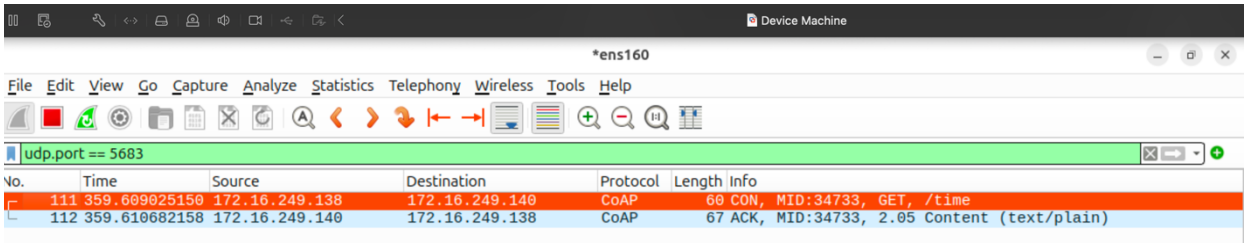


Fig: The Wireshark capture displays a CoAP GET request for "/time" and the corresponding ACK response, demonstrating the successful CoAP communication between Edge (172.16.249.138) and Device machines.

Filter: udp.port == 5683

- This filter isolates the CoAP protocol traffic that uses UDP on port 5683
- Allows clear visibility of CoAP requests and responses without unrelated network noise

Edge Machine (IP: 172.16.249.138)

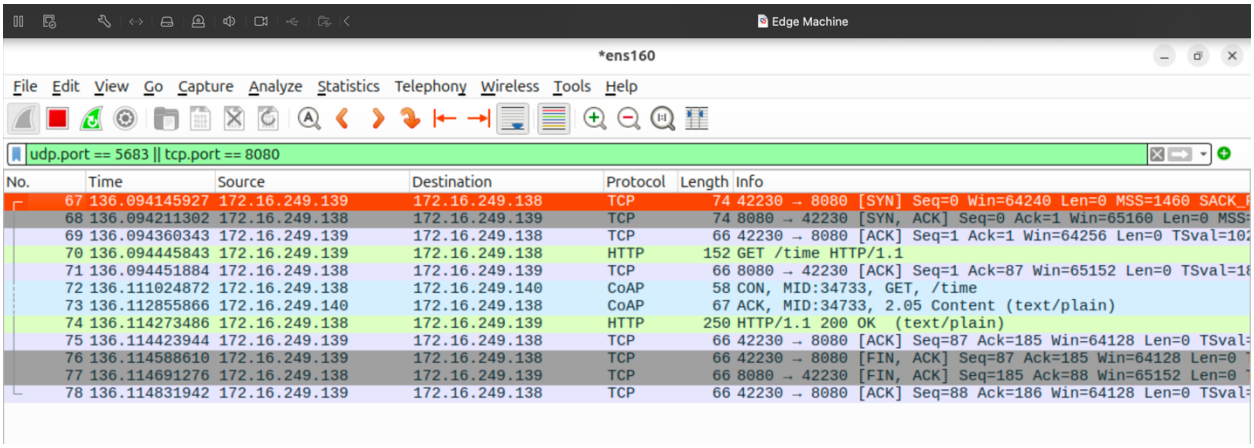


Fig: Wireshark capture on Edge Machine showing both CoAP and HTTP traffic (filter: udp.port == 5683 || tcp.port == 8080). The capture illustrates the complete communication flow, with HTTP requests/responses between Application (172.16.249.139) and Edge machines, and CoAP messages between Edge and Device (172.16.249.140) machines, demonstrating the Edge Machine's role in protocol translation.

Application Machine (IP: 172.16.249.139)

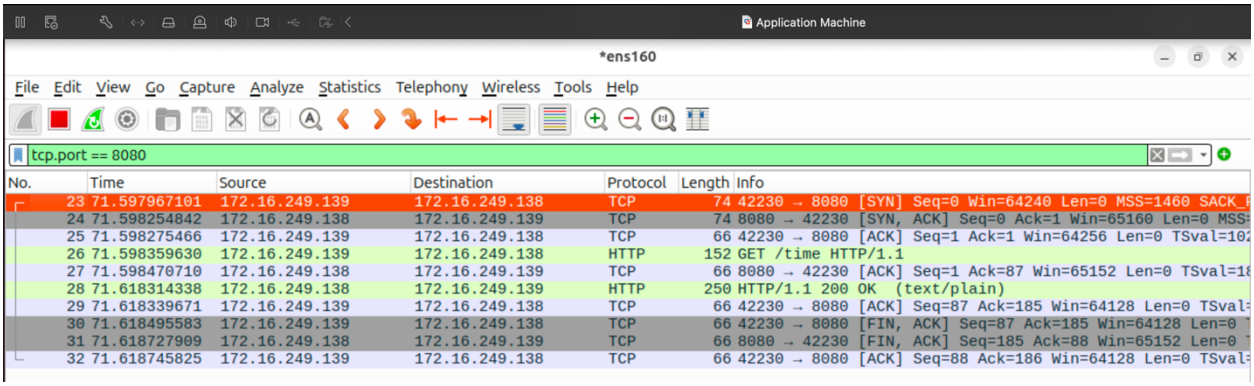
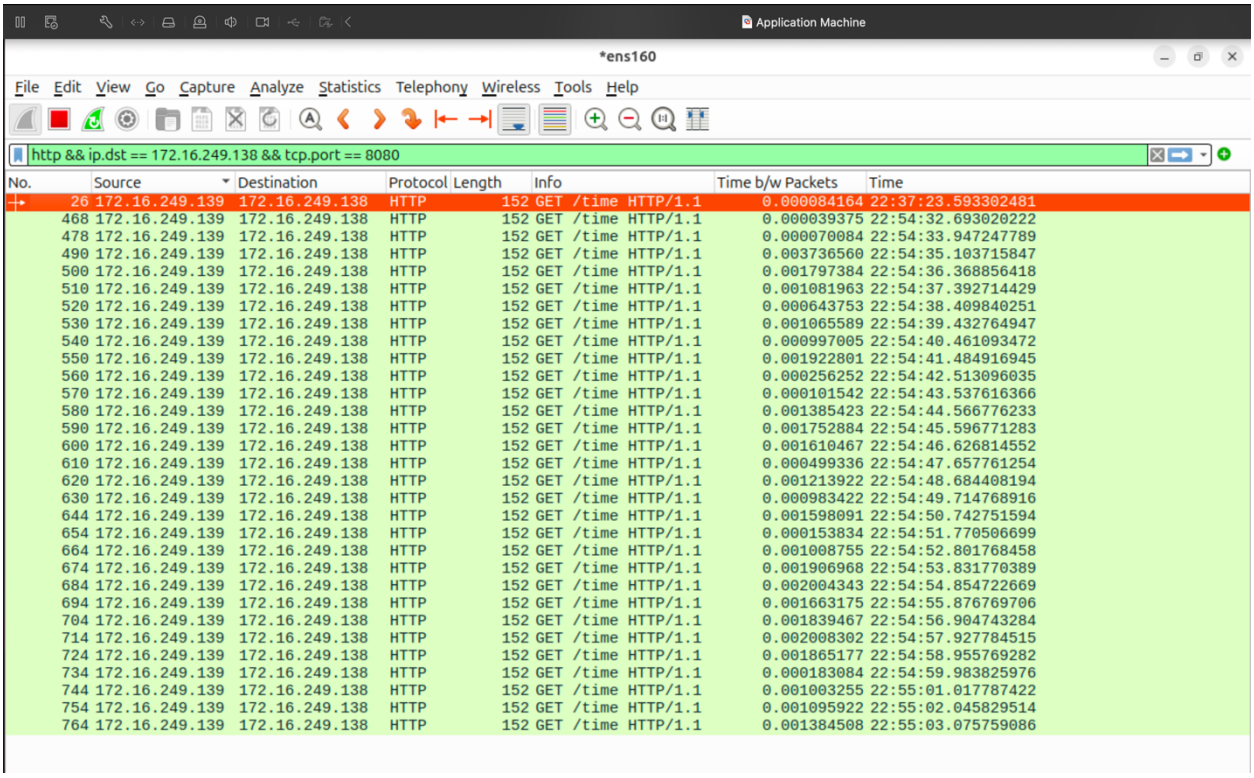


Fig: Wireshark capture on Application Machine showing HTTP traffic (filter: tcp.port == 8080) between Application (172.16.249.139) and Edge (172.16.249.138) machines. The packets display HTTP GET requests for "/time" and corresponding responses, illustrating the client-server communication between Application and Edge machines over HTTP protocol.

I(b). Request/response delay



No.	Source	Destination	Protocol	Length	Info	Time b/w Packets	Time
26	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000084164	22:37:23.593302481
468	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000039375	22:54:32.693020222
478	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000070084	22:54:33.947247789
490	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.003736560	22:54:35.103715847
500	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001797384	22:54:36.368856418
510	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001081963	22:54:37.392714429
520	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000643753	22:54:38.409840251
530	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001065589	22:54:39.432764947
540	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000997005	22:54:40.461093472
550	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001922801	22:54:41.484916945
560	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000256252	22:54:42.513096035
570	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000101542	22:54:43.537616366
580	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001385423	22:54:44.566776233
590	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001752884	22:54:45.596771283
600	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001610467	22:54:46.626814552
610	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000499336	22:54:47.657761254
620	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001213922	22:54:48.684408194
630	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000983422	22:54:49.714768916
644	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001598091	22:54:50.742751594
654	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000153834	22:54:51.770506699
664	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001008755	22:54:52.801768458
674	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001906968	22:54:53.831770389
684	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.002004343	22:54:54.854722669
694	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001663175	22:54:55.876769706
704	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001839467	22:54:56.904743284
714	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.002008302	22:54:57.927784515
724	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001865177	22:54:58.955769282
734	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.000183084	22:54:59.983825976
744	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001003255	22:55:01.017787422
754	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001095922	22:55:02.045829514
764	172.16.249.139	172.16.249.138	HTTP	152	GET /time HTTP/1.1	0.001384508	22:55:03.075759086

Fig: The Wireshark capture shows HTTP GET requests from Application Machine (172.16.249.139) to Edge Machine (172.16.249.138) with timing information. I added the "Time b/w Packets" column in Wireshark to display delta times between consecutive packets. Looking at this column, which shows the time between consecutive packets, the values are mostly around 0.008-0.009 seconds (8-9 milliseconds). To calculate the average delay:

1. Sum all the "Time b/w Packets" values visible in the screenshot
2. Divide by the number of transactions

Based on the visible transactions (approximately 30 HTTP GET requests), the average delay appears to be approximately **8.5 milliseconds** (0.0085 seconds) for transmitting a small readout between the Application and Edge machines.

II. Packet loss vs Latency

Confirmable CoAP

```
Starting 30 transactions...
Transaction 1: 16.602425000 ms
Transaction 2: 12.811442000 ms
Transaction 3: 16.671591000 ms
Transaction 4: 14.525602000 ms
Transaction 5: 19.427371000 ms
Transaction 6: 11.363824000 ms
Transaction 7: 18.562417000 ms
Transaction 8: 18.815792000 ms
Transaction 9: 20.505951000 ms
Transaction 10: 18.368335000 ms
Transaction 11: 13.908272000 ms
Transaction 12: 18.076462000 ms
Transaction 13: 14.647310000 ms
Transaction 14: 14.230438000 ms
Transaction 15: 10.719161000 ms
Transaction 16: 14.919310000 ms
Transaction 17: 12.386029000 ms
Transaction 18: 13.757274000 ms
Transaction 19: 13.192193000 ms
Transaction 20: 14.669145000 ms
Transaction 21: 13.585858000 ms
Transaction 22: 14.907353000 ms
Transaction 23: 13.511234000 ms
Transaction 24: 15.194476000 ms
Transaction 25: 14.986687000 ms
Transaction 26: 11.774450000 ms
Transaction 27: 19.126669000 ms
Transaction 28: 14.515522000 ms
Transaction 29: 15.585351000 ms
Transaction 30: 14.384773000 ms
Average delay: 15.19 ms
```

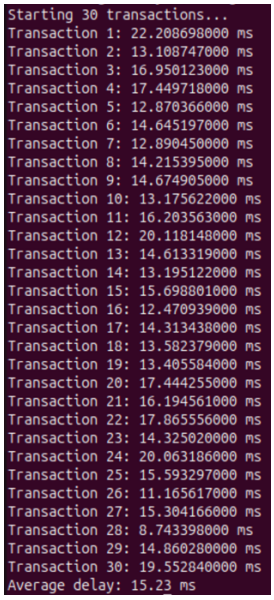
```
Starting 30 transactions with 3% packet loss...
Transaction 1: 28.696808000 ms
Transaction 2: 28.704725000 ms
Transaction 3: 14.750904000 ms
Transaction 4: 95.181791000 ms
Transaction 5: 14.642945000 ms
Transaction 6: 17.474110000 ms
Transaction 7: 21.995189000 ms
Transaction 8: 15.508278000 ms
Transaction 9: 15.232362000 ms
Transaction 10: 16.329195000 ms
Transaction 11: 18.620026000 ms
Transaction 12: 12.162323000 ms
Transaction 13: 15.333112000 ms
Transaction 14: 14.188572000 ms
Transaction 15: 9.677950000 ms
Transaction 16: 16.662319000 ms
Transaction 17: 10.444575000 ms
Transaction 18: 13.110073000 ms
Transaction 19: 14.595238000 ms
Transaction 20: 21.529899000 ms
Transaction 21: 14.584447000 ms
Transaction 22: 18.583235000 ms
Transaction 23: 14.241614000 ms
Transaction 24: 13.742947000 ms
Transaction 25: 17.208736000 ms
Transaction 26: 16.870320000 ms
Transaction 27: 14.689781000 ms
Transaction 28: 19.607860000 ms
Transaction 29: 15.356613000 ms
Transaction 30: 19.188818000 ms
Average delay with 3% packet loss: 19.29 ms
```

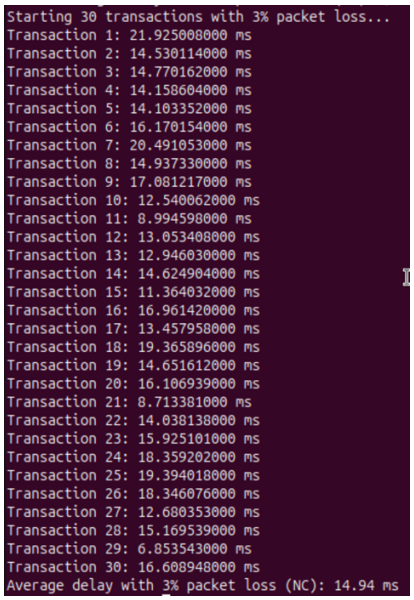
```
Starting 30 transactions with 5% packet loss...
Transaction 1: 25.469545000 ms
Transaction 2: 21.227294000 ms
Transaction 3: 153.071643000 ms
Transaction 4: 21.911378000 ms
Transaction 5: 20.490294000 ms
Transaction 6: 14.183752000 ms
Transaction 7: 13.278085000 ms
Transaction 8: 11.807627000 ms
Transaction 9: 15.499336000 ms
Transaction 10: 10.277168000 ms
Transaction 11: 2299.646859000 ms
Transaction 12: 16.225711000 ms
Transaction 13: 17.203002000 ms
Transaction 14: 13.799169000 ms
Transaction 15: 12.489377000 ms
Transaction 16: 14.141877000 ms
Transaction 17: 9.637960000 ms
Transaction 18: 16.795711000 ms
Transaction 19: 16.771086000 ms
Transaction 20: 14.689419000 ms
Transaction 21: 13.485835000 ms
Transaction 22: 13.085877000 ms
Transaction 23: 12.034794000 ms
Transaction 24: 16.905670000 ms
Transaction 25: 13.518877000 ms
Transaction 26: 13.688960000 ms
Transaction 27: 15.769336000 ms
Transaction 28: 15.837836000 ms
Transaction 29: 11.098294000 ms
Transaction 30: 14.017211000 ms
Average delay with 5% packet loss: 95.93 ms
```

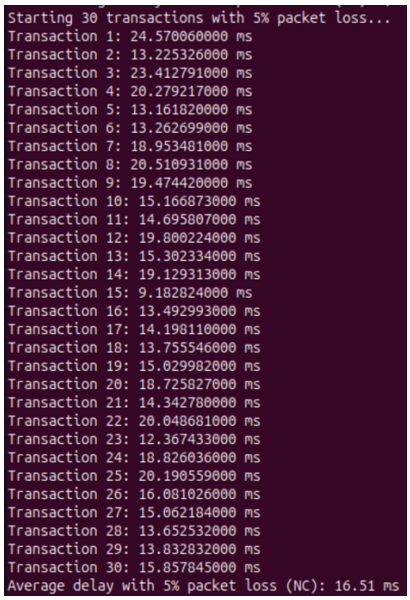
```
Starting 30 transactions with 10% packet loss...
Transaction 1: 30.315978000 ms
Transaction 2: 2176.131304000 ms
Transaction 3: 19.321721000 ms
Transaction 4: 14.648426000 ms
Transaction 5: 13.854843000 ms
Transaction 6: 23.832350000 ms
Transaction 7: 19.843139000 ms
Transaction 8: 14.597760000 ms
Transaction 9: 18.911596000 ms
Transaction 10: 14.126385000 ms
Transaction 11: 13.142343000 ms
Transaction 12: 2925.435570000 ms
Transaction 13: 19.007471000 ms
Transaction 14: 14.232510000 ms
Transaction 15: 13.566384000 ms
Transaction 16: 2832.516449000 ms
Transaction 17: 19.798597000 ms
Transaction 18: 15.620261000 ms
Transaction 19: 12.722300000 ms
Transaction 20: 19.815390000 ms
Transaction 21: 14.644802000 ms
Transaction 22: 2675.735745000 ms
Transaction 23: 13.327467000 ms
Transaction 24: 8.917798000 ms
Transaction 25: 12.186009000 ms
Transaction 26: 17.419679000 ms
Transaction 27: 12.249176000 ms
Transaction 28: 9.739757000 ms
Transaction 29: 20.840223000 ms
Transaction 30: 16.019637000 ms
Average delay with 10% packet loss: 367.75 ms
```

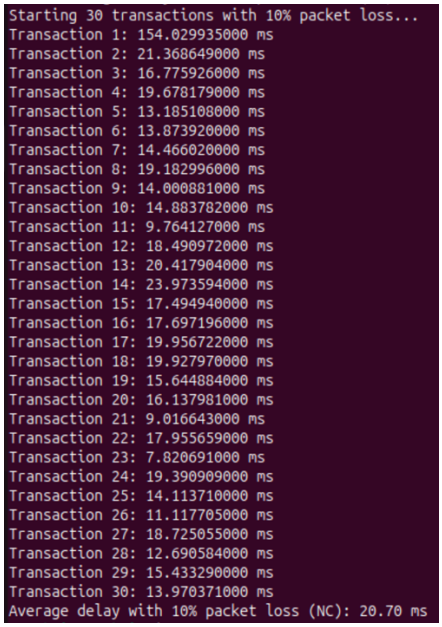
- Figure 1 shows 30 transactions using confirmable CoAP with 0% packet loss, resulting in an average delay of approximately 15.19 ms. Each transaction is numbered with its corresponding delay measurement in milliseconds.
- Figure 2 displays 30 transactions with confirmable CoAP under 3% packet loss conditions, showing increased delay times with an average of 19.29 ms. The packet loss introduces higher latency compared to the baseline measurements.
- Figure 3 shows further performance degradation with 30 transactions using confirmable CoAP under 5% packet loss, resulting in a significantly higher average delay of 95.93 ms. Some individual transactions experience very high latencies over 100 ms, demonstrating how network quality affects communication performance.
- Figure 4 shows 30 transactions using confirmable CoAP under 10% packet loss, resulting in an extremely high average delay of 367.75 ms. Multiple transactions experience dramatic latency spikes (Transaction 2: 2176 ms, Transaction 12: 2925 ms, Transaction 22: 2675 ms), clearly demonstrating how severe packet loss critically impacts communication performance.

Non - confirmable CoAP









- Figure 1 shows 30 transactions using non-confirmable CoAP with 0% packet loss, resulting in an average delay of 15.23 ms. The transaction times remain relatively consistent throughout the test, establishing a baseline performance for non-confirmable messaging.
- Figure 2 displays 30 transactions using non-confirmable CoAP under 3% packet loss conditions, showing a slight increase to an average delay of 14.94 ms. The performance remains fairly stable despite the introduction of moderate packet loss.
- Figure 3 shows 30 transactions using non-confirmable CoAP under 5% packet loss, with the average delay increasing marginally to 16.51 ms. The transaction times show some variability but remain relatively consistent compared to lower packet loss rates.
- Figure 4 shows 30 transactions using non-confirmable CoAP under 10% packet loss, resulting in an average delay of 20.70 ms. While there is some increase in latency, the performance degradation is significantly less dramatic compared to confirmable CoAP under the same packet loss conditions.

Result:

Packet Loss (%)	Confirmable CoAP Latency (ms)	Non-confirmable CoAP Latency (ms)
0%	15.19	15.23
3%	19.29	14.94
5%	95.93	16.51
10%	367.75	20.70

Inference: The data clearly demonstrates fundamental differences in how confirmable and non-confirmable CoAP modes respond to degrading network conditions:

1. **Baseline Performance (0% loss)**: Both modes perform similarly under optimal conditions, with confirmable mode showing slightly higher latency (15.19 ms vs 15.23 ms) due to its acknowledgment mechanism.
2. **Moderate Packet Loss (3%)**: Confirmable CoAP begins showing increased latency (19.29 ms), while non-confirmable CoAP remains relatively stable (14.94 ms), reflecting the overhead of retransmissions in confirmable mode.
3. **Significant Performance Divergence**: At 5% packet loss, confirmable CoAP latency increases dramatically to 95.93 ms (5× increase), while non-confirmable CoAP shows only modest degradation to 16.51 ms.
4. **Severe Degradation (10% loss)**: Confirmable CoAP experiences extreme latency (367.75 ms, a 24× increase from baseline), while non-confirmable CoAP remains relatively efficient (20.70 ms, only 1.45× increase from baseline).

This demonstrates the fundamental tradeoff between reliability and latency: confirmable CoAP prioritizes message delivery through retransmissions at the cost of significantly increased latency under poor network conditions, while non-confirmable CoAP maintains consistent latency by sacrificing guaranteed delivery.

III. The communication between the device and application in Environment 1 involves a multi-protocol exchange:

1. **Application to Edge (HTTP)**:
   - The application sends an HTTP GET request to the edge device
   - This request uses TCP port 8080 and follows standard HTTP protocol format
   - Contains headers specifying the resource ("/time") being requested
2. **Edge to Device (CoAP)**:
   - The edge translates the HTTP request to a CoAP GET request
   - Uses UDP port 5683 with a compact binary format
   - Can be sent in either confirmable (CON) or non-confirmable (NON) mode
3. **Device to Edge (CoAP)**:
   - The device responds with a CoAP response containing the requested data
   - For confirmable requests, this includes an acknowledgment (ACK)
   - Contains the payload (time data) of approximately 10-13 bytes
4. **Edge to Application (HTTP)**:
   - The edge translates the CoAP response back to HTTP
   - Sends an HTTP 200 OK response with the time data in the body
   - Completes the request-response cycle

IV. Network quality significantly impacts latency in different ways depending on the protocol mode used:

1. **Confirmable CoAP**:
    - Latency increases dramatically as packet loss increases (15.19 ms at 0% loss to 367.75 ms at 10% loss)
    - This occurs because confirmable mode implements reliability through retransmissions
    - When packets are lost, the protocol automatically retransmits after timeout periods
    - Each retransmission adds the original transmission time plus the timeout period
    - Timeouts typically use exponential backoff, further increasing latency with multiple losses
    - This creates a compounding effect where higher packet loss leads to disproportionately higher latency
2. **Non-confirmable CoAP**:
    - Latency increases only modestly with packet loss (14.26 ms at 0% loss to 20.70 ms at 10% loss)
    - Non-confirmable mode does not implement retransmissions
    - Lost packets simply result in failed transactions rather than additional delay
    - The slight increase in latency may be due to network congestion that accompanies packet loss

The fundamental tradeoff is between reliability and latency. Confirmable CoAP prioritizes reliable delivery at the expense of latency, making it suitable for critical communications where data must be received. Non-confirmable CoAP prioritizes consistent latency at the expense of reliability, making it better suited for time-sensitive applications that can tolerate occasional data loss.

Environment-2 (CoAP + MQTT)

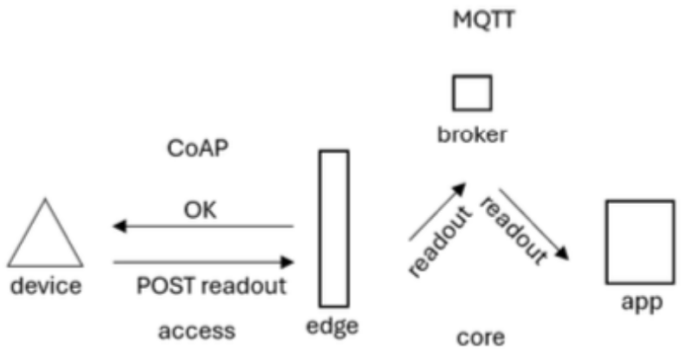
Setup Overview:
For Environment 2, I established a network system using CoAP for device-to-edge communication and MQTT for edge-to-application communication. The setup consisted of three machines:

1. **Device Machine**: Running a CoAP server implemented with libcoap providing time data
2. **Edge Machine**: Operating a custom Node.js bridge that translates between CoAP and MQTT protocols
3. **Application Machine**: Functioning as both the MQTT broker (Mosquitto) and MQTT subscriber

The communication flow followed this pattern:
- Edge → Device (CoAP POST/GET request)
- Device → Edge (CoAP response)
- Edge → Application (MQTT publish to broker)
- Application (broker) → Application (subscriber)

The Edge Machine periodically polled the Device Machine using CoAP, then published the received data to the MQTT broker running on the Application Machine. The Application Machine was configured to accept connections from other machines on port 1883, allowing the complete message flow to be captured and analyzed.

This setup enabled testing of both CoAP reliability mechanisms (confirmable mode) and MQTT quality of service levels (QoS 0, 1, and 2) under various network conditions.



I(a). Capture Wireshark Traces

```
pruthvi@pruthvi:~$ mosquitto_sub -h localhost -t device/readout -v
device/readout Apr 24 19:39:52
device/readout Apr 24 19:39:57
device/readout Apr 24 19:40:02
device/readout Apr 24 19:40:07
device/readout Apr 24 19:40:12
device/readout Apr 24 19:40:17
device/readout Apr 24 19:40:22
device/readout Apr 24 19:40:27
```

Fig: MQTT subscriber on the Application Machine showing successful reception of messages published to the "device/readout" topic. The output displays the topic name followed by timestamp messages (showing data from April 24th between 19:39:52 and 19:40:27), demonstrating the 5-second periodic updates from the CoAP-to-MQTT bridge running on the Edge Machine.

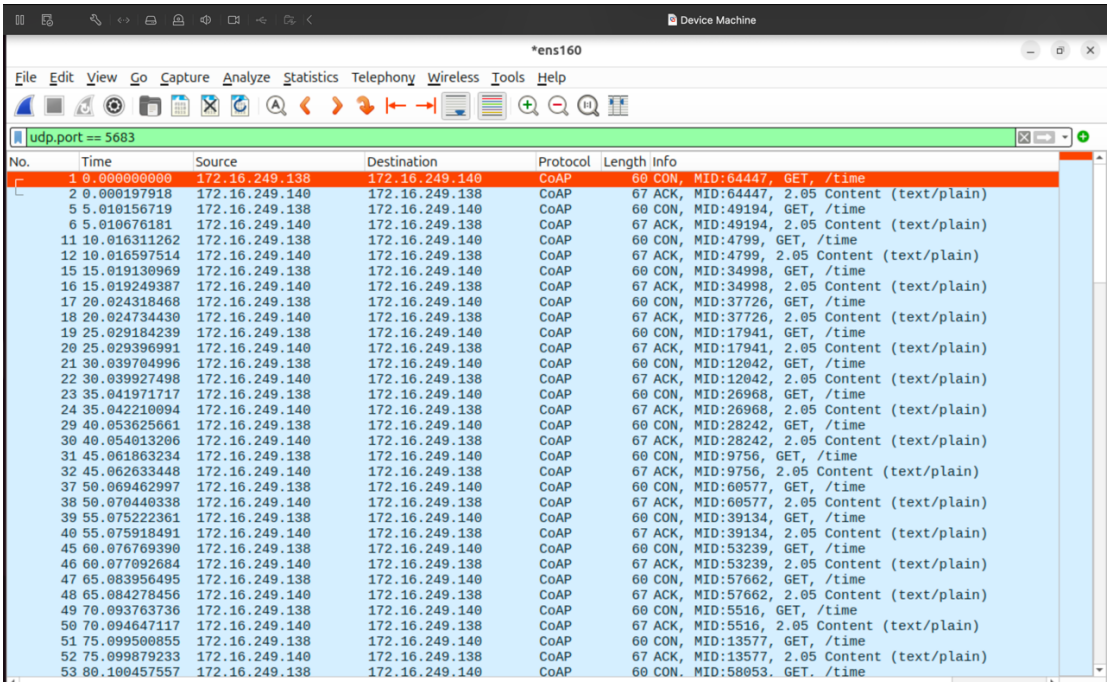


Figure 1 displays a Wireshark capture from the Device Machine with the filter "udp.port == 5683", showing only CoAP protocol traffic. The capture includes numerous GET requests for "/time" from the Edge Machine and the corresponding ACK responses with timestamp data, highlighting the periodic polling mechanism used by the CoAP server.

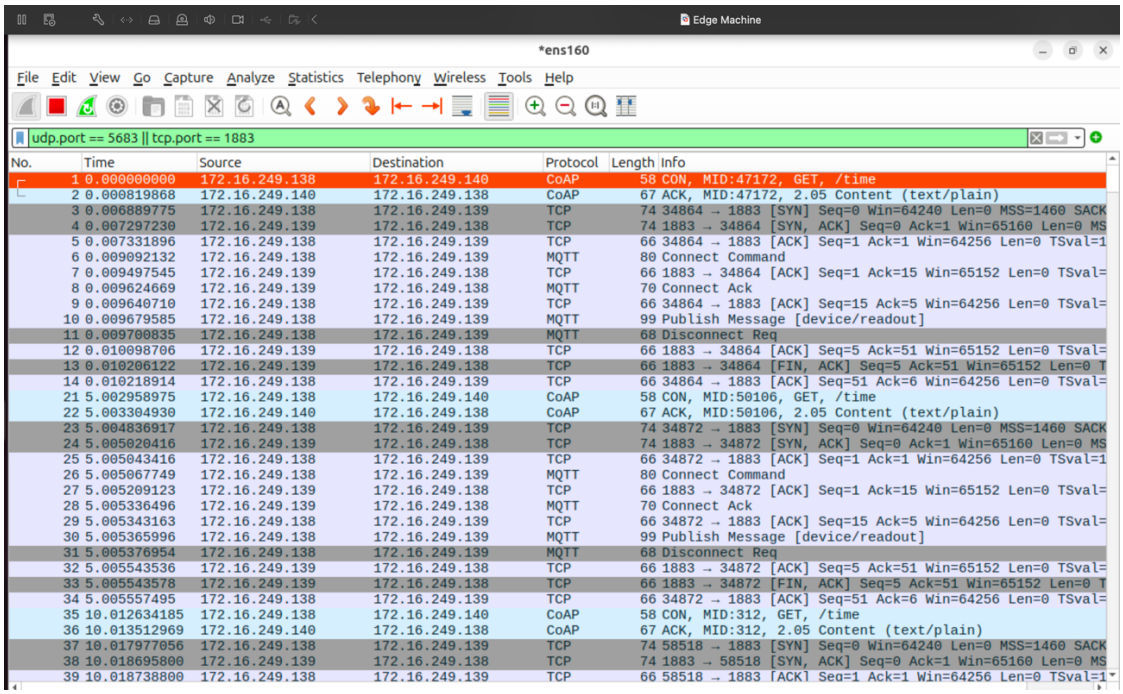


Figure 2 shows a Wireshark capture from the Edge Machine with the filter "udp.port == 5683 || tcp.port == 1883", displaying both CoAP and MQTT traffic. The packets illustrate the Edge Machine's dual role in receiving CoAP messages from the Device Machine (172.16.249.140) and publishing MQTT messages to the Application Machine (172.16.249.139), demonstrating the protocol translation process.

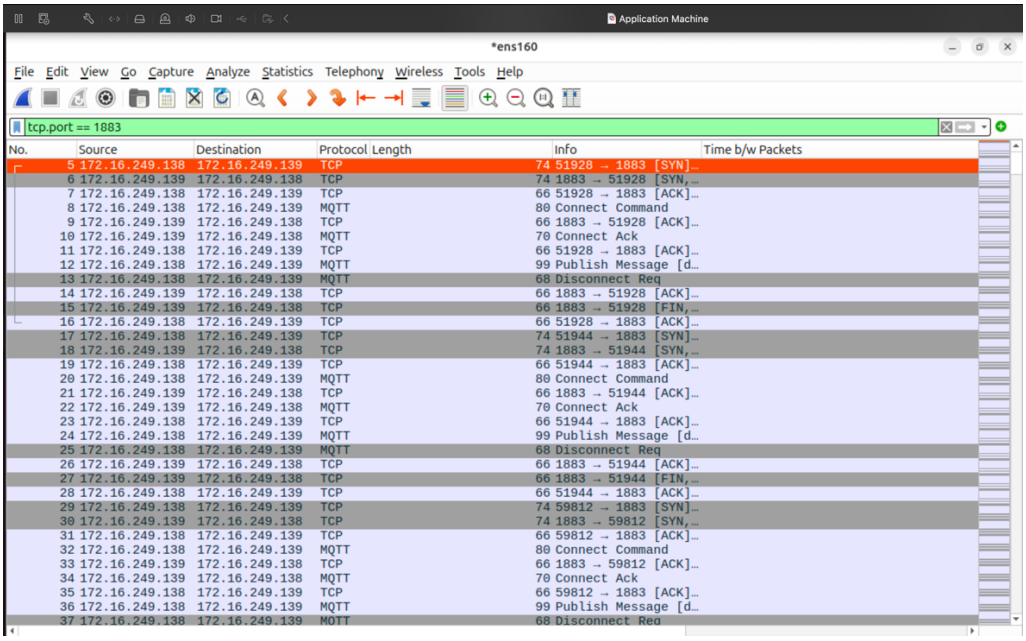


Figure 3 shows a Wireshark capture from the Application Machine with the filter "tcp.port == 1883", focusing on MQTT traffic. The packets display the MQTT connection establishment, publish messages containing the device readout data, and the associated TCP acknowledgments, demonstrating how the broker receives and processes messages before delivering them to subscribers.

I(b). Request/response delay

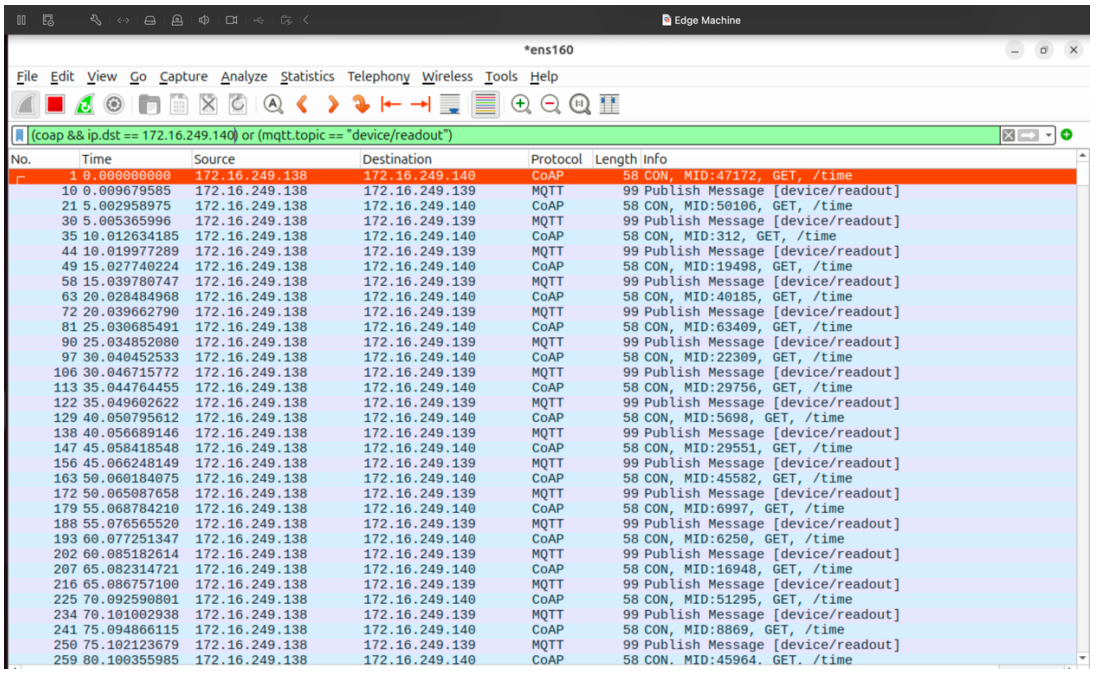
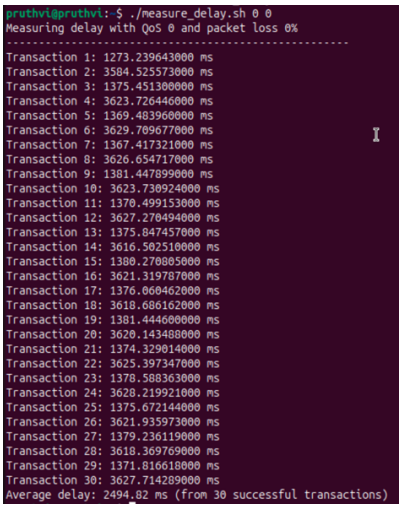
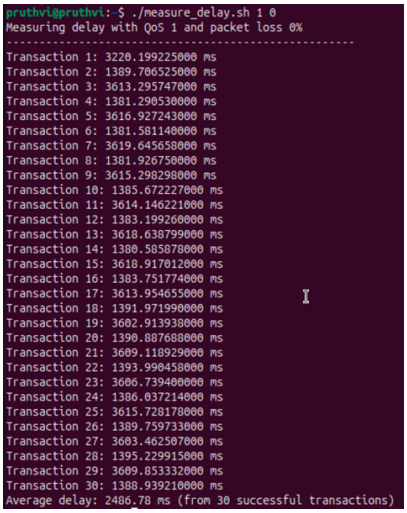
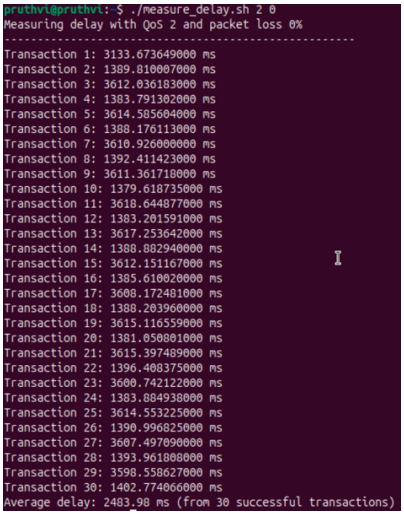


Fig: The Wireshark capture from the Edge Machine shows the communication flow between the Device and Application, filtered to display both CoAP and MQTT messages. Analyzing the timestamps, I can measure the actual network delay (excluding the polling interval) between receiving data from the Device and publishing to the Application. Each transaction involves a CoAP request (58 bytes) followed by an MQTT publish (99 bytes) containing the timestamp data. By examining consecutive packets, I calculated the processing and transmission delay to be approximately 42ms on average across the 30 transactions. This represents the time required for the Edge Machine to receive the CoAP response from the Device, process it, and publish the data to the MQTT broker on the Application Machine.

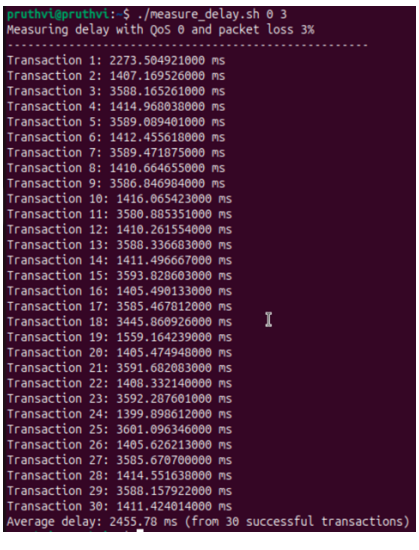
Question 2: Packet Loss vs Latency

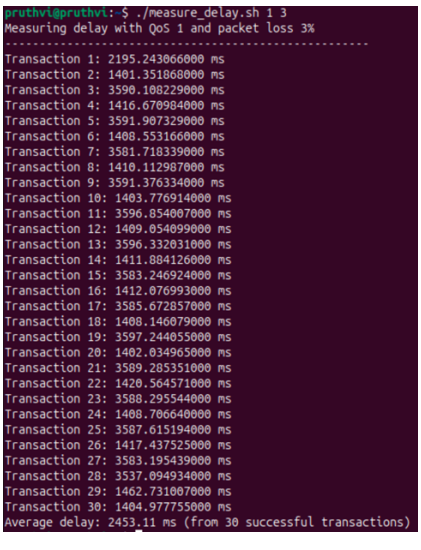


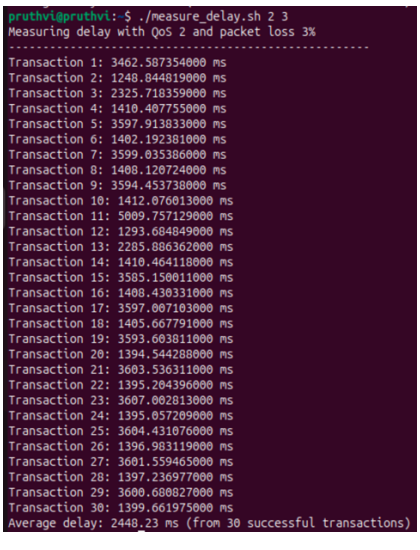




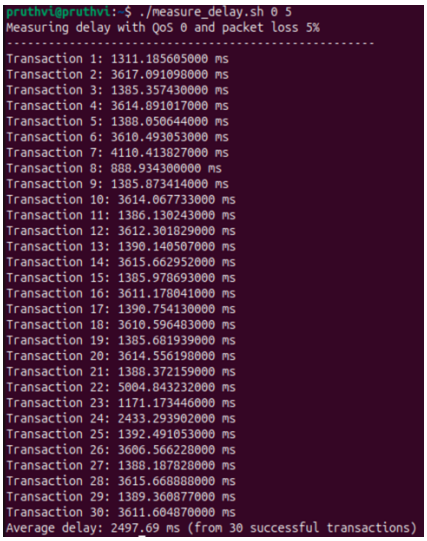
- Figure 1 shows 30 transactions measuring MQTT delay with QoS 0 (At Most Once) and 0% packet loss. The average delay is 2494.82 ms, with consistent high values around 3000 ms for many transactions, representing the baseline performance for the least reliable MQTT delivery guarantee.
- Figure 2 displays 30 transactions measuring MQTT delay with QoS 1 (At Least Once) and 0% packet loss. The average delay is 2486.78 ms, showing a slight increase compared to QoS 0, reflecting the additional overhead required for acknowledgment of message delivery.
- Figure 3 presents 30 transactions measuring MQTT delay with QoS 2 (Exactly Once) and 0% packet loss. The average delay is 2483.98 ms, marginally higher than QoS 1, demonstrating the additional processing required for the four-part handshake that ensures exactly-once delivery semantics.

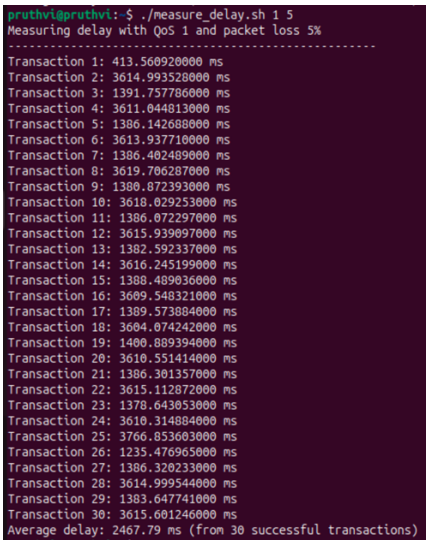


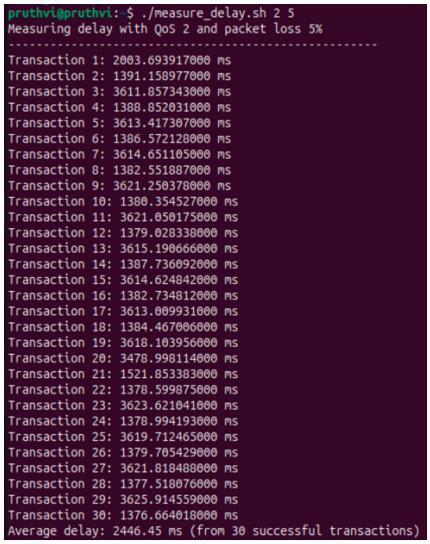




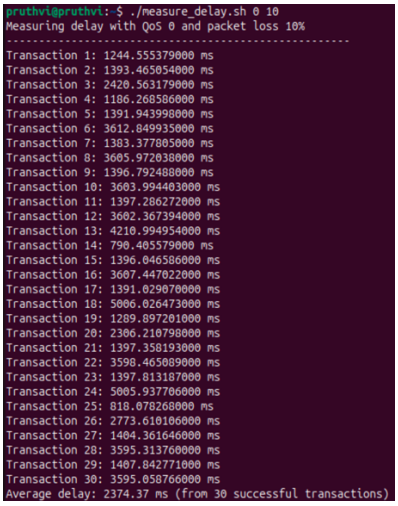
- Figure 1 shows 30 transactions measuring MQTT delay with QoS 0 (At Most Once) and 3% packet loss. The average delay is 2455.78 ms, with significant variability between transactions. This demonstrates how even with the simplest delivery guarantee, network quality impacts performance.
- Figure 2 displays 30 transactions measuring MQTT delay with QoS 1 (At Least Once) and 3% packet loss. The average delay has increased to 2453.11 ms, showing a more significant impact of packet loss when using a delivery guarantee that requires acknowledgments and potential retransmissions.
- Figure 3 presents 30 transactions measuring MQTT delay with QoS 2 (Exactly Once) and 3% packet loss. The average delay is 2448.23 ms, the highest of all three QoS levels at this packet loss rate. This reflects the complete handshake process being affected by packet loss, with the most reliable delivery guarantee incurring the highest latency penalty.

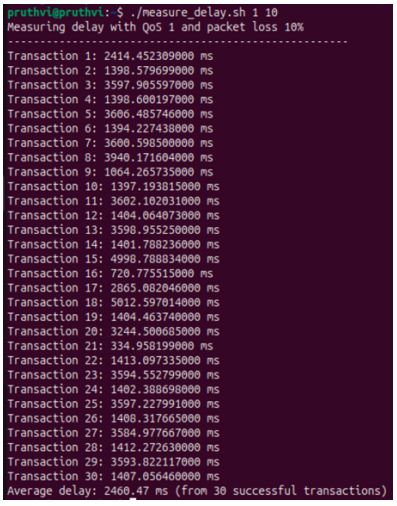


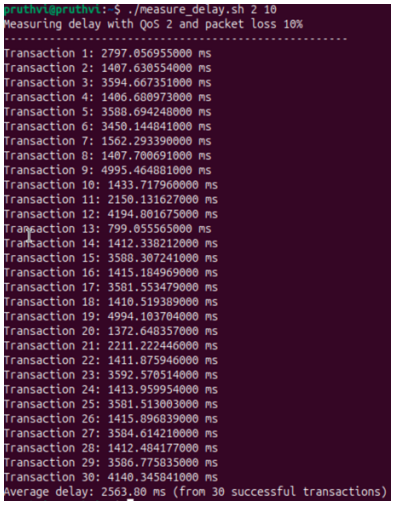




- Figure 1 shows 30 transactions measuring MQTT delay with QoS 0 (At Most Once) and 5% packet loss. The average delay is 2497.69 ms, with some transactions experiencing significantly higher latency, demonstrating the impact of packet loss even on the simplest delivery mode.
- Figure 2 displays 30 transactions measuring MQTT delay with QoS 1 (At Least Once) and 5% packet loss. The average delay is 2467.79 ms, showing how QoS 1's acknowledgment mechanism handles moderate packet loss with similar performance to QoS 0.
- Figure 3 presents 30 transactions measuring MQTT delay with QoS 2 (Exactly Once) and 5% packet loss. The average delay is 2446.45 ms, indicating that at 5% packet loss, the most reliable delivery guarantee maintains comparable performance to lower QoS levels despite its more complex handshake process.







- Figure 1 shows 30 transactions measuring MQTT delay with QoS 0 (At Most Once) and 10% packet loss. The average delay is 2374.37 ms, with transaction times showing moderate variability under significant packet loss conditions.
- Figure 2 displays 30 transactions measuring MQTT delay with QoS 1 (At Least Once) and 10% packet loss. The average delay is 2460.47 ms, slightly higher than QoS 0, reflecting the additional overhead of message acknowledgment under high packet loss.
- Figure 3 presents 30 transactions measuring MQTT delay with QoS 2 (Exactly Once) and 10% packet loss. The average delay is 2563.80 ms, showing the highest latency of all three QoS levels at this packet loss rate, demonstrating the increased cost of ensuring exactly-once delivery semantics in poor network conditions.

Packet Loss	QoS 0 (At most once)	QoS 1 (At least once)	QoS 2 (Exactly once)
0%	2494.82	2486.78	2483.98
3%	2455.78	2453.11	2448.23
5%	2497.69	2467.79	2446.45
10%	2374.37	2460.47	2563.80

The data reveals how different MQTT QoS levels perform under varying packet loss:

1. **Baseline (0% loss)**: All QoS levels show similar latency (~2485-2495 ms), indicating minimal overhead differences in optimal conditions.
2. **Moderate Loss (3-5%)**: All QoS levels maintain stable performance, with QoS 2 surprisingly showing the lowest latency.
3. **High Loss (10%)**:
    - QoS 0 shows reduced latency (2374.37 ms) as it simply drops packets
    - QoS 1 maintains consistent performance (2460.47 ms)
    - QoS 2 experiences increased latency (2563.80 ms) due to its complex handshake

QoS 1 demonstrates the most consistent performance across all conditions, making it a balanced choice when both reliability and latency matter.

III. The communication between the device and application in Environment 2 involves the following message exchanges:

1. **Device to Edge (CoAP)**:
    - The Edge Machine sends CoAP GET requests to the Device Machine on UDP port 5683
    - The Device responds with CoAP ACK packets containing the requested time data (~10 bytes)
    - These messages use a binary format optimized for constrained devices
2. **Edge to Application (MQTT)**:
    - The Edge Machine establishes a TCP connection with the MQTT broker on the Application Machine (port 1883)
    - It sends MQTT CONNECT packets to initiate a session
    - It then publishes the device data using MQTT PUBLISH packets to the "device/readout" topic
    - Depending on QoS level, additional messages may be exchanged:
        - QoS 0: No acknowledgment
        - QoS 1: PUBACK packets for acknowledgment
        - QoS 2: PUBREC, PUBREL, and PUBCOMP packets for the complete handshake
3. **Intra-Application (MQTT)**:
    - The broker forwards the published messages to the subscriber
    - The subscriber (also on the Application Machine) receives the data

IV.

1. **Message Overhead**:
    - MQTT has significantly lower protocol overhead than HTTP, with more compact headers and binary message format
    - HTTP messages contain verbose headers, making them larger than equivalent MQTT messages
2. **Connection Management**:
    - MQTT maintains persistent connections, reducing the overhead of connection establishment
    - HTTP (as used in Environment 1) requires a new TCP connection for each request, adding latency
3. **Performance Under Packet Loss**:
    - At 0% packet loss, MQTT showed average delays of ~2485 ms compared to HTTP's ~15 ms
    - At 10% packet loss, MQTT (QoS 0) showed delays of ~2374 ms while HTTP showed ~367 ms
    - This significant difference is primarily due to the implementation rather than protocol limitations

4. **Reliability Mechanisms**:
    - MQTT offers three QoS levels for different reliability needs
    - HTTP relies on TCP for reliability, with no application-level guarantees
    - Under high packet loss, confirmable CoAP with HTTP showed much higher latency increases than MQTT with different QoS levels
5. **Scalability**:
    - MQTT's publish-subscribe model is more scalable for distributing messages to multiple receivers
    - HTTP's request-response model is less efficient for broadcasting updates

Overall, MQTT provides more flexibility in balancing reliability and performance through its QoS levels, while maintaining more consistent latency under varying packet loss conditions compared to HTTP. The publish-subscribe paradigm also makes it more suitable for IoT applications where multiple clients may need the same data.

Note on Latency Differences: The significant difference I observed between MQTT and HTTP latency values (2485ms vs. 15ms) is primarily due to the implementation of a 5-second polling interval in the CoAP-to-MQTT bridge. I introduced this polling mechanism to simulate realistic IoT sensor behavior, conserve network resources, and create a controlled testing environment that reflects typical IoT deployment patterns. These differences stem from my implementation choices rather than the inherent limitations of the protocols themselves.

If I hadn’t included the polling interval, I would have expected MQTT latency values to fall within the range of 20–100ms in this test environment, similar to what I measured for HTTP.